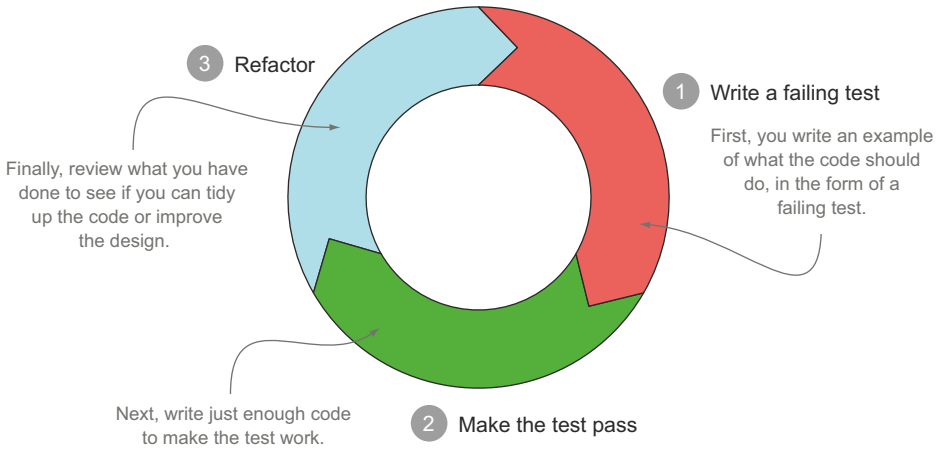


Figure 1.5 Test-Driven Development relies on a simple, three-phase cycle.

For example, suppose Paul is a Java developer working on a new financial trading application in a large bank. He has been asked to implement a new feature to transfer money from one account to another. He creates an `Account` class with a `transfer()` method, a `deposit()` method, and so on. The corresponding unit tests are focused on testing these methods:

```
public class BankAccountTest {
    @Test
    public void testTransfer() {...}

    @Test
    public void testDeposit() {...}
}
```

Tests like this are better than nothing, but they can limit your options. For example, they don't describe what you expect the `transfer()` and `deposit()` functions to do, which makes them harder to understand and to fix if they break. They're tightly coupled to the method they test, which means that if you refactor the implementation, you need to rename your test as well. And because they don't say much about what they're actually testing, it's hard to know what other tests (if any) you need to write before you're done.

North observed that a few simple practices, such as naming unit tests as full sentences and using the word "should," can help developers write more meaningful tests, which in turn helps them write higher quality code more efficiently. When you think in terms of what the class *should* do, instead of what method or function is being tested, it's easier to keep your efforts focused on the underlying business requirements.

For example, Paul could write more descriptive tests along the following lines:

```
public class WhenTransferringInternationalFunds {
    @Test
    public void should_transfer_funds_to_a_local_account() {...}
```